Поведенческие паттерны. Снимок



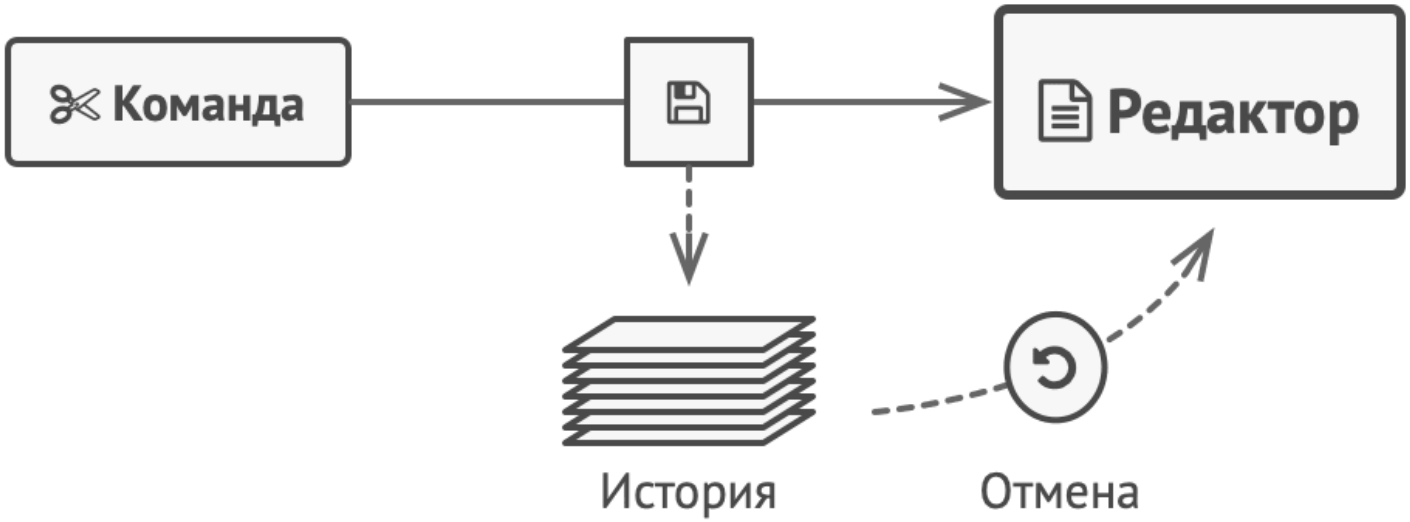
Паттерн Снимок
Memento (Хранитель)



Снимок — это поведенческий паттерн проектирования, который позволяет сохранять и восстанавливать прошлые состояния объектов, не раскрывая подробностей их реализации.

Проблема

Сохранение состояния объектов для возможности отмены операций



Доступ внешних сущностей к полям объекта ограничен при выполнении снимка «извне»

private: не скопировать
public: небезопасно

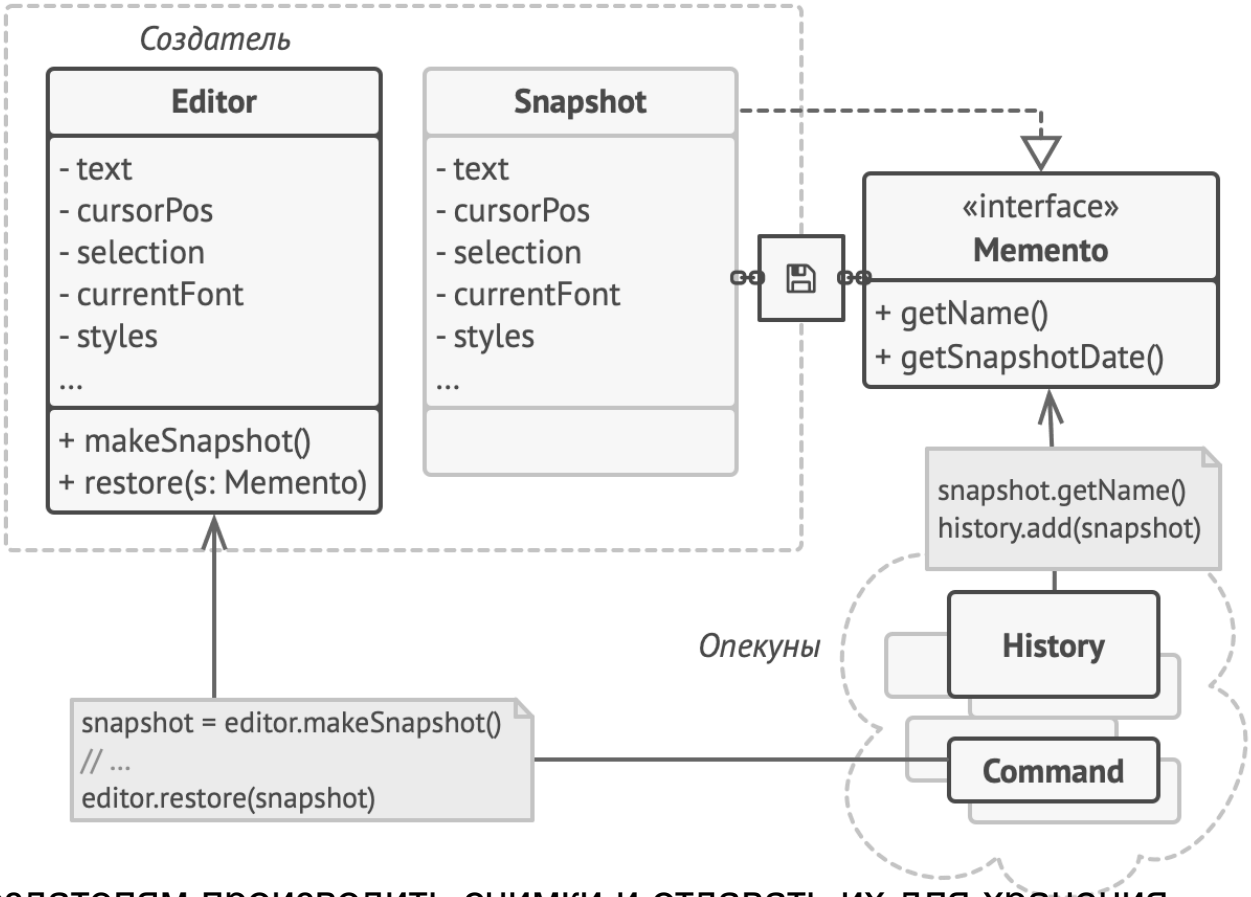


Решение

Паттерн **Снимок** поручает создание копии состояния объекта самому объекту, который этим состоянием владеет.

Паттерн предлагает держать копию состояния в специальном объекте-*снимке* с ограниченным интерфейсом, позволяющим, например, узнать дату изготовления или название снимка.

Снимок должен быть открыт для своего *создателя*, позволяя прочесть и восстановить его внутреннее состояние.



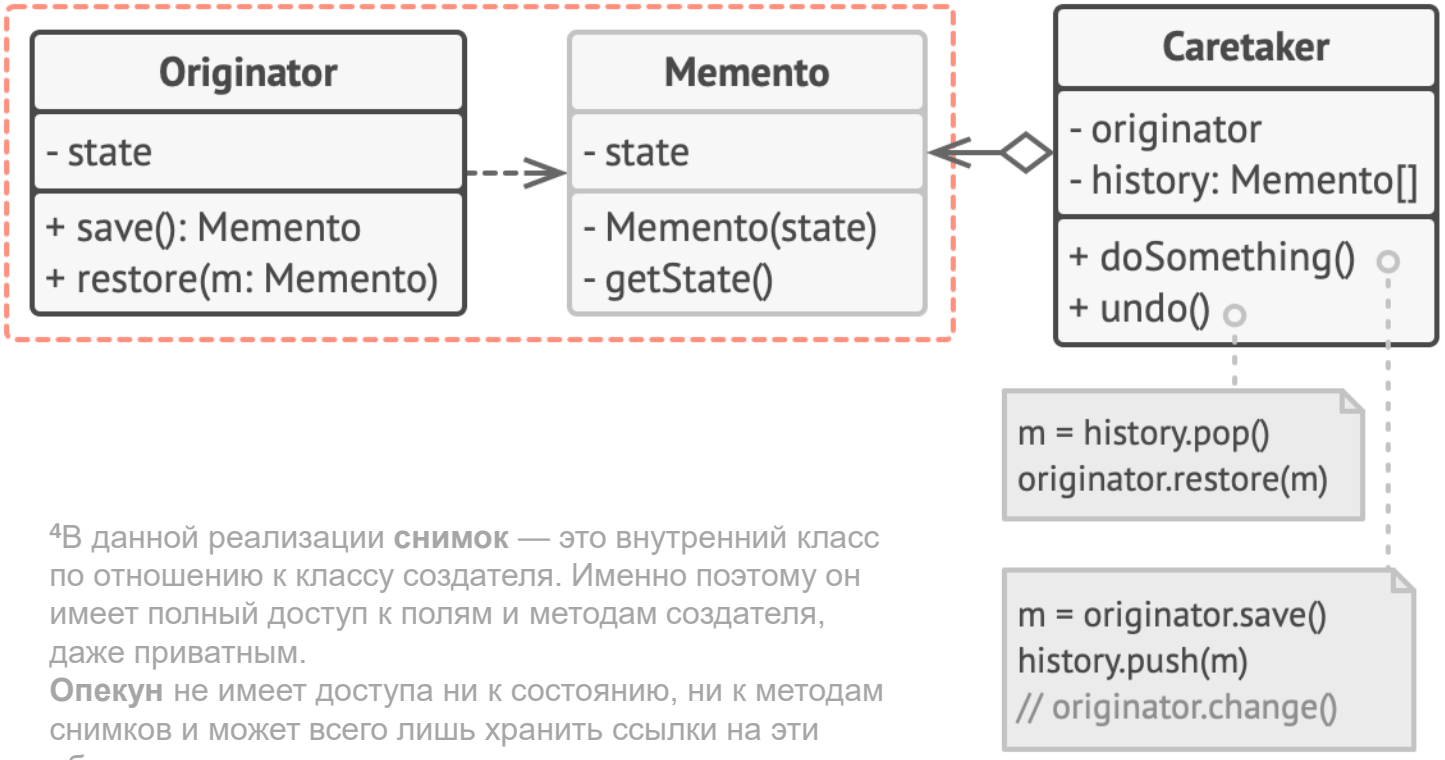
Такая схема позволяет создателям производить снимки и отдавать их для хранения другим объектам, называемым *опекунами*. Опекунам доступен только ограниченный интерфейс снимка, но не закрытый раздел.

Опекун может попросить создателя восстановить своё состояние, передав ему соответствующий снимок.

Классическая реализация на вложенных классах

[1]**Создатель** может производить снимки своего состояния, а также воспроизводить прошлое состояние, если подать в него готовый снимок.

[2]**Снимок** — это простой объект данных, содержащий состояние создателя. Надёжнее всего сделать объекты снимков неизменяемыми, передавая в них состояние только через конструктор.



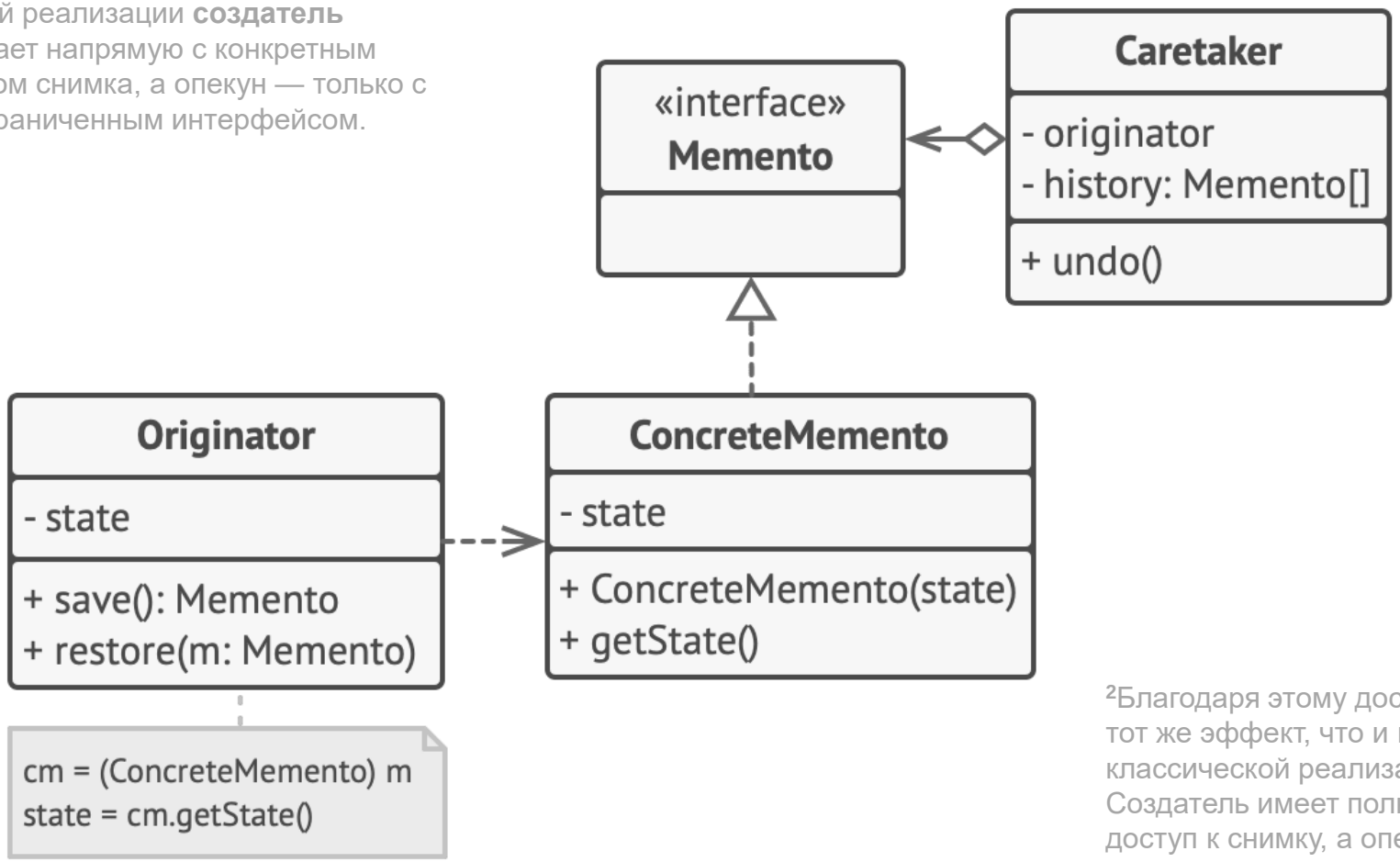
[3]**Опекун** должен знать, когда делать снимок создателя и когда его нужно восстанавливать.

Опекун может хранить историю прошлых состояний создателя в виде стека из снимков. Когда понадобится отменить выполненную операцию, он возьмёт «верхний» снимок из стека и передаст его создателю для восстановления.

[4]В данной реализации **снимок** — это внутренний класс по отношению к классу создателя. Именно поэтому он имеет полный доступ к полям и методам создателя, даже приватным.
Опекун не имеет доступа ни к состоянию, ни к методам снимков и может всего лишь хранить ссылки на эти объекты.

Реализация с пустым промежуточным интерфейсом

[1]В этой реализации **создатель** работает напрямую с конкретным классом снимка, а опекун — только с его ограниченным интерфейсом.

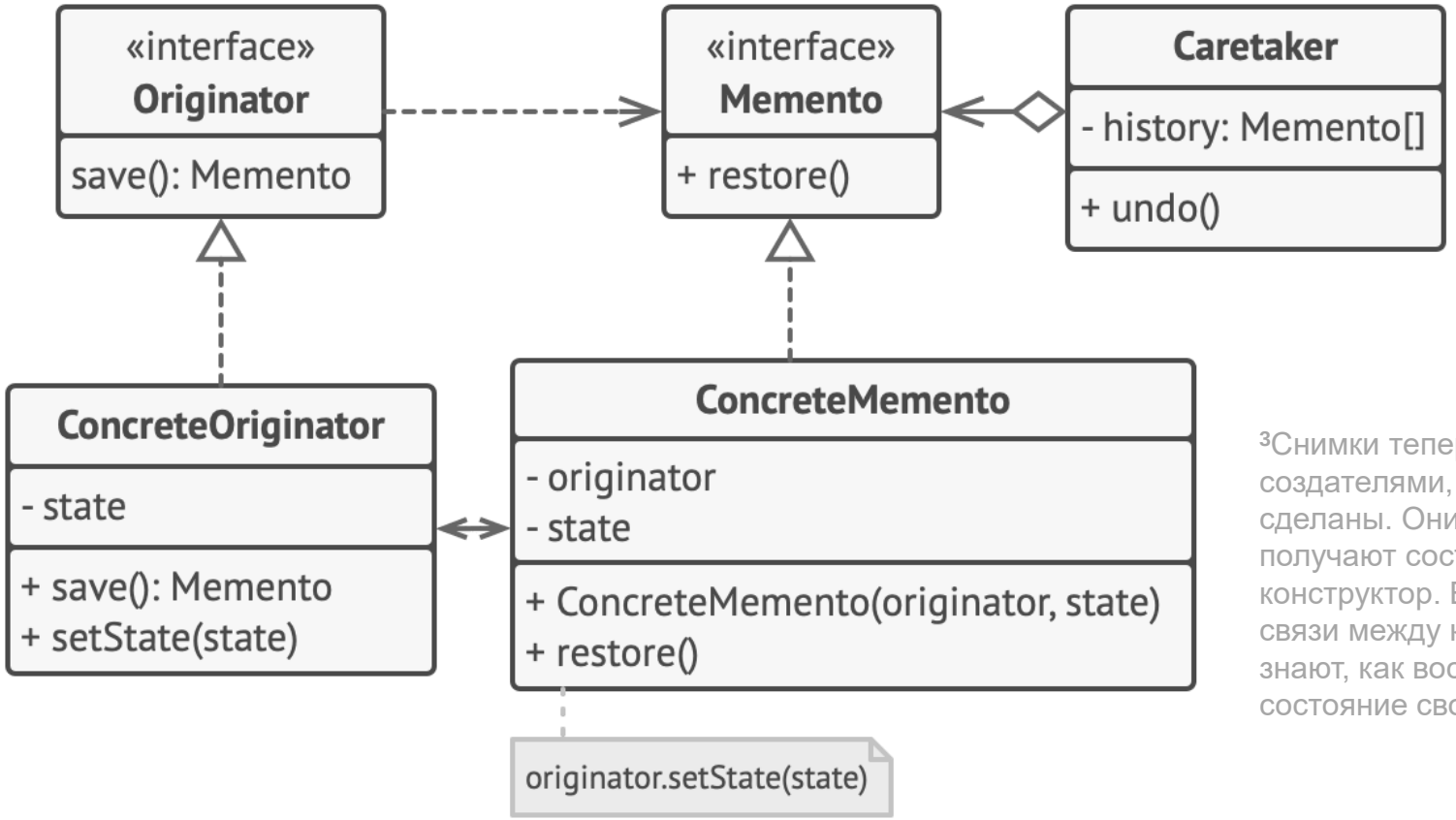


[2]Благодаря этому достигается тот же эффект, что и в классической реализации. Создатель имеет полный доступ к снимку, а опекун — нет.

Снимки с повышенной защитой

[1]Эта реализация разрешает иметь несколько видов создателей и снимков. Каждому классу создателей соответствует свой класс снимков. Ни создатели, ни снимки не позволяют другим объектам прочесть своё состояние.

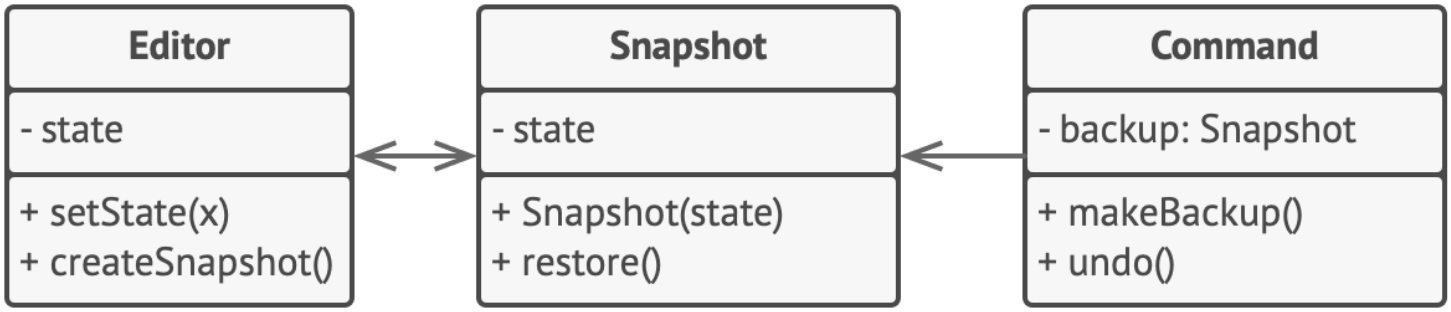
[2]Здесь опекун ещё более жёстко ограничен в доступе к состоянию создателей и снимков. Но, с другой стороны, опекун становится независим от создателей, поскольку метод восстановления теперь находится в самих снимках.



[3]Снимки теперь связаны с теми создателями, из которых они сделаны. Они по-прежнему получают состояние через конструктор. Благодаря близкой связи между классами, снимки знают, как восстановить состояние своих создателей.

Пример

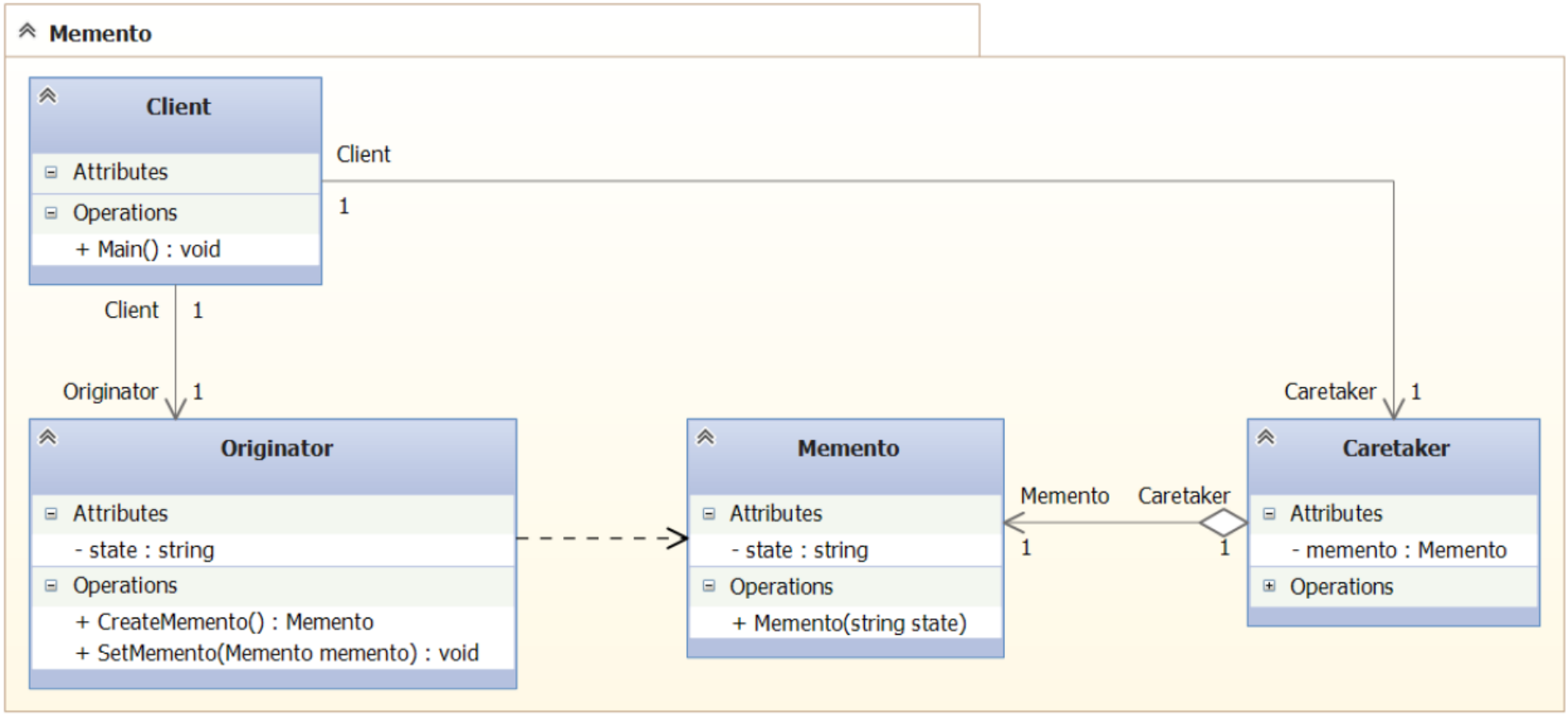
Хранение снимков состояния текстового редактора.



Объекты **команд** выступают в роли *опекунов* и запрашивают *снимки* у редактора перед тем, как выполнить своё действие.

Если потребуется отменить операцию, команда сможет восстановить состояние редактора, используя сохранённый снимок.

Структура паттерна UML



Структура паттерна на языке C#

```
class Program
{
    static void Main()
    {
        Originator originator = new Originator();
        originator.State = "On";

        Caretaker caretaker = new Caretaker();
        caretaker.Memento = originator.CreateMemento();

        originator.State = "Off";

        originator.SetMemento(caretaker.Memento);
    }
}
```

`Caretaker` - **Опекун**:
Отвечает за сохранность объекта-хранителя и не производит над ним никаких действий, и не исследует его внутреннее содержимое.

`Memento` – **Снимок (Хранитель)**:
Хранит в себе состояние объекта-хозяина класса `Originator`. Размер хранимого состояния может быть разный и зависит от потребностей хозяина. Объект-хранитель должен запрещать кому бы то ни было, кроме объекта-хозяина получать доступ к хранимому состоянию.

```
class Originator
{
    public string State { get; set; }

    public void SetMemento(Memento memento)
    {
        State = memento.State;
    }

    public Memento CreateMemento()
    {
        return new Memento(State);
    }
}
```

```
class Memento
{
    public string State
    {
        get;
        private set;
    }

    public Memento(string state)
    {
        this.State = state;
    }
}
```

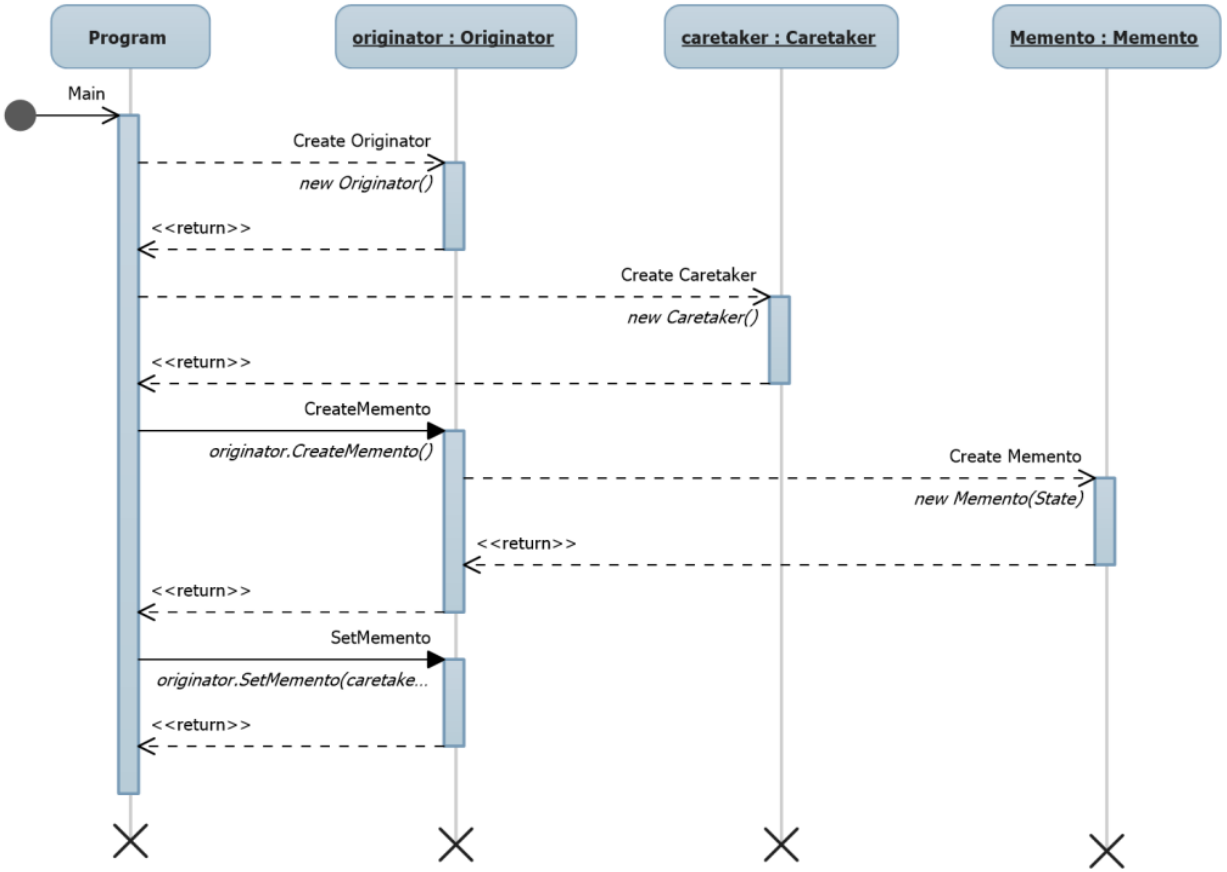
```
class Caretaker
{
    public Memento Memento
    {
        get;
        set;
    }
}
```

`Originator` - **Хозяин**:
Создает объект-снимок и помещает в него свое текущее внутреннее состояние. Впоследствии использует снимок для восстановления своего ранее сохраненного состояния.

Отношения между участниками

Отношения между классами

- Класс Originator связан связью отношения зависимости с классом Memento.
- Класс Caretaker связан связью отношения агрегации с классом Memento.
- Отношения между объектами
- Объект-опекун (Caretaker) получает от объекта-хозяина (Originator) объект-снимок (Memento), некоторое время держит его у себя, а затем возвращает объекту-хозяину.



Применение

1. Когда необходимо сохранять мгновенные снимки состояния объекта (или его части), чтобы впоследствии объект можно было восстановить в том же состоянии.

Паттерн **Снимок** позволяет создавать любое количество снимков объекта и хранить их, независимо от объекта, с которого делают снимок. Снимки часто используют не только для реализации операции отмены, но и для транзакций, когда состояние объекта нужно «откатить», если операция не удалась.

2. Когда прямое получение состояния объекта раскрывает приватные детали его реализации, нарушая инкапсуляцию.

Паттерн предлагает изготовить снимок самому исходному объекту, поскольку ему доступны все поля, даже приватные.

Шаги реализации

1. **Определить класс создателя**, объекты которого должны создавать снимки своего состояния.

2. **Создать класс снимка** и описать в нём все те же поля, которые имеются в оригинальном классе-создателе.

3. **Сделать объекты снимков неизменяемыми**. Они должны получать начальные значения только один раз, через свой конструктор.

4. **Если ЯП это позволяет, сделать класс снимка вложенным в класс создателя**. Если нет, реализовать для класса снимка пустой интерфейс, который будет доступен остальным объектам программы.

5. **Добавить в класс создателя метод получения снимков**. Создатель должен создавать новые объекты снимков, передавая значения своих полей через конструктор.

6. **Добавить в класс создателя метод восстановления из снимка**.

7. **Опекуны должны знать о том, когда запрашивать снимки** у создателя, где их хранить и когда восстанавливать.

8. **Связь опекунов с создателями можно перенести внутрь снимков**. В этом случае каждый снимок будет привязан к своему создателю и должен будет сам восстанавливать его состояние. Это будет работать либо если классы снимков вложены в классы создателей, либо если создатели имеют соответствующие сеттеры для установки значений своих полей.

Преимущества и недостатки

+ Не нарушает инкапсуляции исходного объекта.

+ Упрощает структуру исходного объекта. Ему не нужно хранить историю версий своего состояния.

- Требует много памяти, если клиенты слишком часто создают снимки.

- Может повлечь дополнительные издержки памяти, если объекты, хранящие историю, не освобождают ресурсы, занятые устаревшими снимками.